

Guía de Evaluación Práctica:

Programación Orientada a Objetos

- **Asignatura:** Programación Orientada a Objetos
- **Fecha de Publicación:** Miércoles 27 de Mayo de 2026
- **Plazo de Entrega:** Jueves 28 de Mayo de 2026, 17:00 hrs
- **Formato de Entrega:** Repositorio GitHub (Código fuente + Diagrama UML)

1. Contexto del Problema

Para mantener un control interno de todas las personas que participan en las distintas actividades de la convención internacional de cómics (**Comic-Con**), se requiere una aplicación en Java que permita registrar a los expositores, sus categorías temáticas, las agrupaciones comunitarias y los paneles de presentación programados.

2. Requerimientos del Sistema (Clases Mínimas)

1. Estructura de Herencia (Clase Madre y Clases Hijas)

- **Persona (Clase Abstracta):** Es la clase base del sistema. Posee los atributos nombre, paísOrigen, edad y una lista de categorías a las que pertenece.
 - **Métodos:** Debe permitir agregar categorías, eliminar categorías y un método abstracto llamado `public abstract void mostrarInformacion();`.

Debido a las características del evento, existen dos tipos específicos de personas que heredan de la clase madre:

- **Cosplayer (Clase Hija):** Posee además los atributos `cantidadTrajes` y `estiloPreferido` (del tipo de dato Enum *EstiloCosplay*). Debe implementar obligatoriamente el método `mostrarInformacion()` detallando todos sus datos.
- **Artista (Clase Hija):** Posee además los atributos `marcaTableta` y `cantidadArtbooksPublicados`. Debe implementar obligatoriamente el método `mostrarInformacion()` detallando todos sus datos.

2. Uso de Interfaces

Para evaluar habilidades especiales de los expositores, se debe implementar la siguiente interfaz:

- **Interactable (Interfaz):** Define el comportamiento de aquellos que interactúan directamente con el público. Posee el método `public void realizarPresentacion();`.
 - o La clase **Cosplayer** debe implementar esta interfaz (el método imprimirá un mensaje estimando su tiempo de preparación según la complejidad del traje).
 - o La clase **Artista** debe implementar esta interfaz (el método imprimirá un mensaje mostrando su portafolio y libros publicados).

3. Uso de Enumeraciones (Enum)

- **EstiloCosplay (Enum):** Define los estilos permitidos para los trajes. Debe contener los valores: ANIME, COMICS, VIDEOJUEGOS, PELICULAS.

4. Clase Categoría

Representa las áreas temáticas de la convención (ej., Manga, Star Wars, Desarrollo Indie).

- **Atributos:** nombre, `numeroCuposMaximos` y `popularidadEstimada`.

- **Métodos:** Debe poder mostrar toda su información.

5. Clase Agrupación

Representa a los grupos temáticos o comunidades cerradas (ej., un club de fans oficial).

- **Atributos:** nombreGrupo, una lista de personas que la componen y una categoría principal asignada.
- **Métodos:** Debe poder agregar personas, eliminar personas y mostrar la información de la agrupación.

6. Clase Panel

Representa una charla o debate programado en el evento.

- **Atributos:** Categoría asociada, una lista de panelistas (que acepta tanto personas individuales como agrupaciones completas mediante polimorfismo) y la horaPresentacion.
- **Métodos:** Debe poder agregar panelistas, eliminar panelistas y mostrar la información del panel.

3. Relaciones y Reglas de Negocio (UML)

Al diseñar tu solución, debes respetar estrictamente las siguientes relaciones de los Juegos Panamericanos:

- **Muchos a Muchos (\$*..*\$):** Una persona pertenece a una o más categorías, mientras que una categoría puede ser asignada a muchas personas.
- **Agregación/Composición:** Una agrupación se compone de personas. Una persona puede pertenecer a una o más agrupaciones, pero también puede tener **cero o una** agrupación opcional registrada en su perfil individual (para quienes exponen solos pero también forman parte de una comunidad).
- **Dependencia/Asociación:** Un panel está estrictamente relacionado a una categoría. La categoría puede existir de forma independiente, pero el panel no puede crearse sin una categoría definida.
- Si un panel se cancela o elimina de la programación, la relación de asistencia de las personas asociadas a ese panel también debe eliminarse.

4. Clase Principal (Main)

Debes construir una clase Main que actúe como punto de entrada del programa. En ella deberás:

1. Crear instancias de categorías, enums, personas específicas (**Cosplayer** y **Artista**), agrupaciones y paneles.
2. Asociar a las personas a sus respectivas categorías y agrupaciones.
3. Invocar ordenadamente los métodos creados, incluyendo el método realizarPresentacion() de la interfaz, para demostrar el uso correcto de la herencia y el polimorfismo mediante la consola.

5. Tareas a Realizar

1. **Modelo UML (40%):** Diseña un diagrama de clases UML que incluya la totalidad de las clases, la clase abstracta, la interfaz, el enum, sus atributos (con visibilidad correcta), métodos con firmas completas y las relaciones correspondientes.
2. **Construcción de Clases (20%):** Implementa las clases en Java respetando el encapsulamiento (uso de private, protected, getters y setters).

3. **Implementación de Código (30%):** Desarrollar la lógica interna de los métodos, polimorfismo, el uso de interfaces y el manejo de colecciones (List o ArrayList) para agregar/eliminar elementos.
4. **Buenas Prácticas (10%):** Código limpio, modular, nombres de variables autoexplicativos y validaciones básicas.